

Flujo y arquitectura de WILDECINEMAS

Guía para explicar cómo se conectan pantallas, servicios y Supabase

Usuario	Componente	Servicio	Supabase	Estado visual
---------	------------	----------	----------	---------------

La aplicación está construida con Angular en el navegador y Supabase como backend. Angular presenta las pantallas y conserva el estado visual. Supabase autentica usuarios, almacena datos y ejecuta operaciones críticas como reservar una butaca, confirmar una compra o validar un QR.

Idea principal del flujo

Cada acción sigue el mismo recorrido: el usuario pulsa un botón o completa un formulario; el componente recibe esa acción; un servicio consulta o modifica datos; Supabase responde; el componente actualiza una señal; Angular vuelve a dibujar únicamente la parte de la vista que depende de esa señal.

Ejemplo: al seleccionar una butaca, ButacasComponent llama a ButacasService. Ese servicio pide a Supabase reservarla. Si Supabase confirma la reserva, el componente agrega la butaca a su señal *seleccionadas* y el resumen se actualiza.

Estructura de navegación

Ruta	Componente que se crea	Objetivo
/	InicioComponent	Catálogo, cartelera, próximos estrenos y ranking.
/login	AuthComponent	Login y registro.
/pelicula/:id	DetallePeliculaComponent	Ficha, funciones y reseñas.
/funciones/:id/butacas	ButacasComponent	Mapa, reservas y compra.
/ticket/:id	TicketComponent	Comprobante, QR y PDF.
/mis-peliculas	MisPeliculasComponent	Historial, alertas y cancelaciones.
/admin	AdminComponent	Mantenimiento y reportes.

Dependencias reales

Los componentes dependen de servicios mediante inyección. Un servicio creado con *providedIn: root* es compartido: no se crea una copia por cada pantalla. Esto permite que el usuario autenticado y su perfil sean los mismos en toda la aplicación.

PeliculasCrudService	PeliculasService	AuthService	Cliente Supabase
----------------------	------------------	-------------	------------------

La cadena anterior muestra el caso del CRUD de películas. El componente administrativo no habla directamente con la base. Usa PeliculasCrudService, que conserva el formulario y la lista; ese servicio usa PeliculasService, que traduce los datos; y PeliculasService usa el cliente entregado por AuthService.

Servicio	Depende de	Responsabilidad
AuthService	Supabase Auth y tabla perfiles	Sesión, perfil, rol, cliente Supabase y vencimiento local.
PeliculasService	AuthService	Lectura y escritura de películas.
PeliculasCrudService	PeliculasService	Estado compartido del formulario y listado administrativo.
ProgramacionService	AuthService	Salas, precio base y funciones.
ButacasService	AuthService	Mapa, reservas temporales y canal Realtime.
ComprasService	AuthService	Confirmación de compra y comprobante.
ReseniasService	AuthService	Promedios, consulta y guardado de reseñas.
ExperienciaClienteService	AuthService	Historial, alertas, ranking y cancelaciones.

Cómo se comunican los componentes

- **Rutas y parámetros:** se transportan identificadores entre pantallas. Por ejemplo, el ID de una función viaja en la URL hasta ButacasComponent.
- **Inputs y outputs:** se usan entre componentes padre e hijo. LoginComponent y RegistroComponent emiten datos; AuthComponent decide qué hacer con ellos.
- **Servicios inyectados:** se usan para operaciones, datos y estado compartido.
- **Signals:** guardan listas, errores, cargas y selecciones. Cuando una señal cambia, Angular actualiza la plantilla.

Flujo de autenticación y roles

LoginComponent	AuthComponent	AuthService	Supabase Auth	perfiles
----------------	---------------	-------------	---------------	----------

Inicio de sesión

- LoginComponent valida que email y contraseña estén presentes y los emite.
- AuthComponent recibe las credenciales y llama a AuthService.login().
- AuthService ejecuta signInWithPassword en Supabase Auth.
- Con el usuario devuelto, AuthService consulta la tabla perfiles para cargar nombre, rol, puntos y crédito.
- AuthService actualiza currentUser y currentProfile. La interfaz reacciona a esos valores.
- AuthComponent navega a /admin si isAdmin() es verdadero; en los demás casos navega al inicio.

Registro

- RegistroComponent valida el formulario reactivo y emite CredencialesRegistro.
- AuthService.register() crea la cuenta en Supabase Auth y guarda los metadatos.
- Un trigger de Supabase crea el perfil asociado en la tabla perfiles.
- Si Supabase crea una sesión, AuthService carga el perfil y registra el comienzo de la sesión local.

Guards y permisos

Guard	Ruta	Decisión
authGuard	/mis-peliculas	Permite entrar solo si existe currentUser.
adminGuard	/admin	Permite entrar solo si currentProfile tiene rol admin.
staffGuard	/validar-qr	Permite entrar a admin o empleado.

Cada guard espera `auth.whenReady()` antes de decidir. Así evita redirigir mientras Supabase todavía está recuperando una sesión previa.

Flujo público: Inicio, detalle y reseñas

Inicio

InicioComponent consulta en paralelo cartelera y próximos estrenos con PeliculasService. Luego pide ranking de ventas a ExperienciaClienteService y resúmenes de reseñas a ReseniasService. Si hay sesión, consulta también las alertas activas del usuario.

Las películas se guardan en señales. Los filtros de título y género no hacen nuevas consultas: una señal calculada filtra la lista ya cargada. Cada resultado se entrega a TarjetaPeliculaComponent como input. La tarjeta solo presenta información y navega al detalle.

Detalle de película

DetallePeliculaComponent toma el id de la URL. Con ese valor pide la película a PeliculasService y las funciones futuras a ProgramacionService. Si la persona elige una función, el componente navega a /funciones/:id/butacas.

Reseñas

ReseniasPeliculaComponent recibe peliculald como input del detalle. Espera que AuthService esté listo, solicita las reseñas y busca si una de ellas pertenece al usuario conectado. Al guardar, usa ReseniasService.guardar(), que realiza un upsert: crea la reseña si no existe o actualiza la reseña del mismo usuario.

Flujo de selección de butacas

Detalle	ButacasComponent	ButacasService	RPC de reserva	Realtime
---------	------------------	----------------	----------------	----------

- ButacasComponent toma el ID de función de la URL.
- Consulta en paralelo la función y el mapa de butacas vendidas.
- Crea un token único para esta compra mediante crypto.randomUUID().
- Cuando se toca una butaca, llama a tomar_reserva_butaca con función, butaca y token.
- Supabase confirma o rechaza la reserva temporal. Solo después de confirmarla se agrega a seleccionadas.
- Supabase Realtime avisa los cambios en reservas_butacas. Además, la pantalla consulta disponibilidad cada 15 segundos.
- Al abandonar la pantalla, ngOnDestroy libera las reservas asociadas a ese token.

Confirmación de compra y ticket

ButacasComponent	ComprasService	RPC de compra	Orden y entradas	Ticket
------------------	----------------	---------------	------------------	--------

Validaciones previas en la interfaz

- No permite más de ocho butacas.
- Calcula VIP como precio base multiplicado por 1,5.
- Bloquea películas próximas sin preventa activa.
- Comprueba la clasificación ATP, +13 o +18 y solicita confirmación cuando corresponde.

La operación definitiva

Al pulsar Comprar, ButacasComponent envía al ComprasService el ID de función, las butacas, el token de reserva y los datos de clasificación. ComprasService llama a la RPC `comprar_orden_con_comprobante`. La RPC es la autoridad: debe comprobar que las reservas existan, pertenezcan al token, sigan vigentes y puedan transformarse en una orden.

Supabase devuelve un comprobante con orden, QR, película, función, sala, formato, idioma, butacas, subtotal, descuento y total. El componente guarda ese comprobante en `sessionStorage` y también lo entrega a la siguiente ruta mediante `history.state`.

Ticket

TicketComponent busca primero el comprobante en `history.state`. Si la navegación no lo tiene, intenta recuperarlo desde `sessionStorage`. Genera el QR y permite exportar PDF. Actualmente no consulta la orden de nuevo a Supabase: por eso abrir el enlace del ticket desde otra sesión puede no mostrar la información.

Historial, alertas y cancelaciones

MisPelículasComponent está protegido por `authGuard`. Después de confirmar que existe una sesión, consulta las funciones del historial y las alertas. Al cancelar una orden, llama a `cancelar_mi_orden` mediante `ExperienciaClienteService`. Luego refresca el perfil con `AuthService.refreshCurrentProfile()` y vuelve a cargar el historial para reflejar el crédito actualizado.

Flujo del panel administrador

AdminComponent se crea únicamente si adminGuard autoriza la ruta. Mantiene una señal llamada seccionActiva y usa @switch para crear el módulo que corresponde a la pestaña elegida. Cada módulo carga sus datos cuando se inicia.

Módulo	Dependencia principal	Flujo
Películas	PeliculasCrudService	Formulario y listado comparten estado. El servicio valida y usa PeliculasService para persistir.
Salas y butacas	ProgramacionService	RPC crea sala y butacas. Otra RPC elimina solo salas sin funciones.
Funciones	PeliculasService + ProgramacionService	Carga películas, precio y funciones. La RPC asigna una sala compatible.
Configuración	ProgramacionService	Consulta y actualiza el precio base de entradas mediante RPC.
Usuarios y roles	AuthService.client.functions	Llama Edge Function crear-usuario-privilegiado.
Reportes	AuthService.client.rpc	Consulta ventas, pendientes, ranking y auditoría.
Candy Bar	Sin módulo funcional	La interfaz muestra que está en preparación.

Caso especial: CRUD de películas

FormularioPeliculaComponent y ListadoPeliculasComponent comparten PeliculasCrudService. Cuando se pide editar, el listado guarda la película elegida en el servicio y emite un evento para que el padre muestre el formulario. Cuando el formulario guarda correctamente, emite otro evento y el padre muestra el listado. El estado real de la edición permanece en el servicio.

Qué hace Angular y qué hace Supabase

Angular y componentes	Supabase y backend
Muestra pantallas, formularios, errores y estados de carga.	Autentica usuarios y mantiene tokens.
Navega entre rutas y pasa IDs por URL o estado de navegación.	Guarda películas, perfiles, reseñas, salas, funciones, reservas y órdenes.
Calcula valores visuales como subtotal y filtros.	Ejecuta reglas críticas mediante RPC.
Mantiene signals para actualizar la interfaz.	Evita conflictos de butacas y valida compras.
Escucha cambios para reflejarlos en pantalla.	Publica cambios de reservas mediante Realtime.

Puntos para explicar al profesor

- La aplicación separa interfaz, acceso a datos y reglas de negocio para que cada parte tenga una responsabilidad clara.
- Los servicios evitan repetir consultas y permiten reutilizar operaciones desde varias pantallas.
- Las señales actualizan la interfaz cuando cambia el estado sin manipular el HTML manualmente.
- Las operaciones sensibles se resuelven en Supabase mediante RPC, porque la validación no debe depender solamente del navegador.
- Realtime se usa para que la disponibilidad de butacas se actualice mientras varias personas compran al mismo tiempo.
- Los guards controlan la navegación según sesión y rol; el backend debe mantener también sus propias políticas y validaciones.

Estado actual de la arquitectura

La mayor parte de la lógica está separada en servicios. Reportes, validación de QR y creación de usuarios todavía llaman de manera directa a `AuthService.client` desde sus componentes. Una mejora futura sería crear `ReportesService`, `ValidacionQrService` y `UsuariosAdminService` para mantener el mismo nivel de separación que ya existe en películas, programación, butacas y compras.

Fin del documento